

Project Description

The Order Management System (OMS) is a robust, full-stack backend application designed to manage the complete lifecycle of e-commerce operations. Built with modern technologies, it facilitates seamless customer registration, product catalog management, order placement, delivery tracking, and administrative analytics. This system ensures efficient handling of product inventory, order statuses, customer data, and business insights while enforcing rules such as stock validation, duplicate prevention, and role-based access control. It supports secure operations for admins, customers, and delivery agents, making it scalable for online businesses.

Key Features

- Customer Registration & Order Placement: Customers can register, browse products, and place orders with real-time stock checks.
- View Order History and Status: Real-time display of placed orders, total amounts, and delivery statuses.
- Admin Dashboard for Order & Product Management: Role-based admin panel to manage products, orders, and customer queries.
- Add/Edit/Remove Products and Customers: Full CRUD operations for products and customer profiles.
- Place Order with Validation: Secure order creation with stock availability and duplicate prevention.
- Prevent Duplicate Orders: Built-in checks to avoid multiple orders for the same item in one session.
- Cancel or Update Order: Customers and admins can cancel or modify orders with approval workflows.
- Get Product-Wise Order List: Query and export lists of orders per product.
- Calculate Total Order Value: Automatic computation of order.

Core Modules

- Home Page: Overview of the store, including products.
- Dashboards: Role-based dashboards for:
 - Admin: Manage products, orders, customers, and delivery assignments.
 - Customers: View profile, order history, and track deliveries.

Customer & Order Workflow

- Customers register or log in via email and password.
- Browse products and add to cart.
- Place order with payment simulation and stock deduction.
- Receive order confirmation and tracking updates.
- Admins approve, process, and assign delivery.

Admin Functionalities

- Add & Edit Products, Categories, and Stock Levels.
- Approve or cancel customer orders.

Dependencies and Technologies Used

Backend Core Technologies

- TypeScript: Strong typing for maintainable and scalable code.
- Express.js: Lightweight server and REST API framework.
- MongoDB: NoSQL database with Mongoose for flexible schema modeling.
- Cloudinary: Cloud-based media storage for product images.
- Zod: Runtime schema validation for request payloads.

Developer Utilities

- Environment Variables: Via dotenv for configuration management.
- Linting and Formatting: ESLint and Prettier for code quality.
- Development Server: ts-node-dev for hot reloading during development.

Authentication & Security

- jsonwebtoken (JWT): Token-based authentication with access/refresh tokens.
- cors: Secure cross-origin resource sharing.
- bcrypt: Secure password hashing with salt rounds.
- http-status: Standardized HTTP response codes.

Frontend Core Technologies

- Next.js (v15.4.1): React framework with App Router and server components.
- React (v19.1.0): Component-based UI library.
- TypeScript: Type-safe frontend development.

State Management & Forms

- React Hook Form (v7.60.0): Performant form handling with validation.
- Zod (v4.0.5): Schema validation integrated with forms.
- @hookform/resolvers (v5.1.1): Zod resolver for React Hook Form.

UI Components & Styling

- Tailwind CSS (v4): Utility-first CSS for rapid, responsive design.
- shadcn/ui: Accessible, customizable UI components.
- Lucide React (v0.525.0): Modern icon set.
- Sonner (v2.0.6): Beautiful toast notifications.

Prerequisites

- Node.js (v16 or higher)
- npm or yarn
- MongoDB (Local or Atlas)

Installation

- Clone the Backend Repository:
`git clone https://github.com/your-username/order-management-backend.git`
`cd order-management-backend`
- Install Dependencies:

npm install

- Set Up Environment Variables: Create a .env file in the root directory:

Environment

NODE_ENV=development

PORT=5000

Database

DATABASE_URL=your-mongodb-uri

Security

BCRYPT_SALT_ROUND=12

JWT Secrets

JWT_ACCESS_SECRET=your-access-token-secret

JWT_REFRESH_SECRET=your-refresh-token-secret

JWT_ACCESS_EXPIRES_IN=1d

JWT_REFRESH_EXPIRES_IN=7d

Cloudinary

CLOUDINARY_CLOUD_NAME=your-cloud-name

CLOUDINARY_API_KEY=your-api-key

CLOUDINARY_API_SECRET=your-api-secret

Nodemailer

FROM_EMAIL=your-sender@gmail.com

FROM_PASS=your-app-password

Frontend URL

CLIENT_URL=http://localhost:3000

- Clone Frontend Repository:

git clone https://github.com/your-username/order-management-frontend.git

cd order-management-frontend

- Install Dependencies:

npm install

- Environment Variables: Create .env.local:

NEXT_PUBLIC_BASE_API=http://localhost:5000/api/v1

NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=your-cloud-name

Backend Scripts

Run these via npm run <script> in the backend directory:

Script	Description
start:dev	Start development server with hot reload
build	Compile TypeScript to JavaScript
start	Start production server
lint	Check code linting issues
lint:fix	Auto-fix linting issues
format	Format code with Prettier
format:fix	Auto-fix formatting issues

Example: Start dev server → npm run start:dev (runs on http://localhost:5000).

Frontend Scripts

Run these via npm run <script> in the frontend directory:

Script	Description
dev	Start development server
build	Build for production
start	Start production server
lint	Run ESLint

Example: Start dev server → npm run dev (runs on http://localhost:3000).

API Endpoints Overview

All endpoints are prefixed with /api/v1. Authentication is required for protected routes (via JWT in Authorization header).

Auth (/auth)

Method	Path	Description	Auth Required
POST	/login	Customer/Admin login	No

Customers (/customer)

Method	Path	Description	Auth Required
POST	/register	Register customer	No
GET	/	Get all customers	Yes (ADMIN)

Products (/product)

Method	Path	Description	Auth Required
POST	/	Create product	Yes (ADMIN)
GET	/	Get all products	No
GET	/:id	Get product by ID	No
PATCH	/:id	Update product	Yes (ADMIN)
DELETE	/:id	Delete product	Yes (ADMIN)
GET	/customers/:id	Get customers who ordered	Yes (ADMIN)

Orders (/order)

Method	Path	Description	Auth Required
POST	/	Create new order	Yes (CUSTOMER)
GET	/	Get all orders	Yes (ADMIN)
GET	/:id	Get order by ID	Yes (ADMIN/CUSTOMER)
GET	/customer/:customerId	Get orders by customer	Yes (ADMIN/CUSTOMER)
PATCH	/:id/status	Update order status	Yes (ADMIN)
DELETE	/:id	Delete order	Yes (ADMIN)

Delivery (/delivery)

Method	Path	Description	Auth Required
PATCH	/:id	Update delivery status	Yes (ADMIN)

Dashboard (/dashboard)

Method	Path	Description	Auth Required
GET	/	Get admin analytics	Yes (ADMIN)

CORE SERVICES & DETAILED IMPLEMENTATION

The backend follows a modular MVC pattern. Each module contains:

- controller.ts – HTTP layer
- service.ts – Business logic
- model.ts – Mongoose schema
- validation.ts – Zod validation
- interface.ts – TypeScript types
- route.ts – Express routes

1. AUTH SERVICE (modules/auth)

Purpose: Secure JWT-based authentication.

- POST /login
→ Validates email/password → bcrypt compare → generates access

2. CUSTOMER SERVICE (modules/customer)

- POST /register
→ Zod validation → checks email uniqueness → hashes password → creates customer
- GET /
→ Admin-only → returns all customers with pagination

3. PRODUCT SERVICE (modules/product)

- POST / → Admin → uploads image to Cloudinary → creates product
- GET / → Public → lists available products
- GET /:id → Public → single product
- PATCH /:id → Admin → partial update
- DELETE /:id → Admin → removes product
- GET /customers/:id → Admin → returns customers who ordered this product

4. ORDER SERVICE (modules/order)

- POST / → Customer → validates stock → deducts inventory → creates order
- GET / → Admin → all orders
- GET /:id → Owner/Admin → order details
- GET /customer/:id → Owner/Admin → customer order history
- PATCH /:id/status → Admin → updates status (restores stock if cancelled)

- DELETE /:id → Admin → deletes order

5. DELIVERY SERVICE (modules/delivery)

- PATCH /:id → Admin → updates tracking status
→ If "Delivered", auto-updates order status

6. DASHBOARD SERVICE (modules/dashboard)

- GET / → Admin only
→ Returns:
 - Total revenue
 - Orders by status
 - Top 5 products

Conclusion

The Order Management System (OMS) is a modern, secure, and scalable solution that simplifies e-commerce operations for businesses. It leverages cutting-edge technologies like TypeScript, Express, MongoDB, and Next.js to deliver a fast, reliable, and user-friendly platform. With role-based access, real-time updates, and comprehensive admin tools, it empowers businesses to manage orders efficiently from placement to delivery.